SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 10

Name of Student : R SHIVA SAI REDDY
Enrollment No. : 2303A51542
Batch No. : 22

Task Description -1(Error Detection and Correction)
Use AI to analyze a Python script and correct all syntax and logical errors.
Sample Input Code:
def calculate_total(nums)
sum = 0 for n
in nums sum
+= n
return total
Expected Output-1:
Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

Prompt:generate a python program to correct the code syntax and logical errors in python syntax and print the output to be corrected and executable code with docstring and comments.

Code:-

```python
# Task-1
# generate a python program to correct the code syntax and logical errors in
def calculate_total(nums):
    """
    Calculate the total sum of numbers in a list.

    Args:
        nums: A list of numbers to sum.

    Returns:
        The sum of all numbers in the list.
    """
    # Avoid using 'sum' as variable name since it shadows built-in function
    total = 0
    for n in nums:
        total += n
    return total

# Test the function
if __name__ == "__main__":
    numbers = [10, 20, 30, 40, 50]
    result = calculate_total(numbers)
    print(f"Total sum: {result}")
```

Output:-

```
Total sum: 150
```

Justification: The original code had a syntax error due to using 'sum' as a variable name, which is a built-in function in Python. By renaming it to 'total', we avoid this conflict and ensure the code runs correctly. Additionally, the function now includes a docstring that explains its purpose, arguments, and return value, improving code readability and maintainability and the test case demonstrates how to use the function and verifies its correctness.

Task Description -2(Code Style Standardization)
Use AI to refactor Python code to comply with standard coding style guidelines.
Sample Input Code: def
findSum(a,b):return a+b
print(findSum(5,10))
Expected Output-2:
Well-structured, consistently formatted Python code following standard style conventions.

Prompt:-
generate a python program to comply the standard coding Style and well formatted code with proper indentation, spacing, and naming conventions to enhance code readability and maintainability.

Code:-

```python
# Task-2
# generate a python program to comply t
def find_sum(a, b):
    """
    Find the sum of two numbers.

    Args:
        a: The first number.
        b: The second number.

    Returns:
        The sum of the two numbers.
    """
    return a + b

print(find_sum(5, 10))
```

Output:-

```
15
```

Justification:
The code is now compliant with standard coding style, including proper indentation, spacing, and naming conventions. The function name 'find_sum' is descriptive of its purpose, and the parameters 'a' and 'b' are clear. The docstring provides a clear explanation of the function's purpose, its arguments, and its return value, enhancing code readability and maintainability. The test case demonstrates how to use the function and verifies its correctness.

Task Description -3(Code Clarity Improvement)
Use AI to improve code readability without changing its functionality.
Sample Input Code:
def f(x,y): return x-y*2
print(f(10,3))
Expected Output-3:
Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity. Prompt:-
generate a python program to improve the code efficiency and performance and functionality
Code:-

```python
# Task-3
# generate a python program to improve the code efficiency and performance and functionality
def calculate_adjusted_difference(base_value, multiplier):
    """
    Calculate the difference between a base value and twice a multiplier.

    Args:
        base_value: The initial value to subtract from.
        multiplier: The value to multiply by 2 before subtracting.

    Returns:
        The result of base_value minus (multiplier * 2).
    """
    return base_value - multiplier * 2

# Test the function
if __name__ == "__main__":
    result = calculate_adjusted_difference(10, 3)
    print(f"Adjusted difference: {result}")
```

Output:-

```
Adjusted difference: 4
```

Justification:
The original code had a syntax error due to using 'sum' as a variable name, which is a built-in function in Python. By renaming it to 'total', we avoid this conflict and ensure the code runs correctly. Additionally, the function now includes a docstring that explains its purpose, arguments, and return value, improving code readability and maintainability and the test case demonstrates how to use the function and verifies its correctness.

Task Description -4(Structural Refactoring)
Use AI to refactor repetitive code into reusable functions.
Sample Input Code: print("Hello Ram") print("Hello Sita") print("Hello Ravi")

Prompt:-

generate a python program using reusable functions with modular design to break down the code into smaller, reusable functions that perform specific tasks, to improve code organization and maintainability.

Code:-

```
# Task-4
# generate a python program using r
def print_greeting(name):
    print(f"Hello {name}")

print_greeting("Ram")
print_greeting("Sita")
print_greeting("Ravi")
```

Output:-

```
Hello Ram
Hello Sita
Hello Ravi
```

Justification:
The code is now organized into a reusable function 'print_greeting' that takes a name as an argument and prints a greeting message. This modular design allows for better code organization and maintainability, as the greeting logic is encapsulated within a single function that can be easily reused with different names. The test cases demonstrate how to use the function with various inputs, verifying its functionality.

Task Description -5(Efficiency Enhancement) Use AI to optimize Python code for better performance.
Sample Input Code:
numbers = [ ] for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers))
Expected Output-5:
Optimized Python code that achieves the same result with improved performance.

Prompt:generate a python program to optimize the code with improve the performance and efficiency by using appropriate data structures, algorithms, and built-in functions to optimize the code's performance and efficiency, and show the time complexity of the optimized code.

Code:-

```python
# Task-5
# generate a python program to optimize the code with improve the performance
def generate_squares(n):
    """
    Generate squares of numbers from 1 to n-1 using a generator.
    Args:
        n: The upper limit (exclusive) for generating squares.
    Returns:
        A generator object that yields squared values.
    Time Complexity: O(1) for creation, O(n) for full iteration
    Space Complexity: O(1) - uses generator instead of storing all values
    """
    return (i * i for i in range(1, n))
# Optimized approach using generator
if __name__ == "__main__":
    # Generator approach - memory efficient
    squares_gen = generate_squares(500000)
    count = sum(1 for _ in squares_gen)
    print(f"Count of squares: {count}")
    # Alternative: Using list comprehension (if storage needed)
    # squares = [i * i for i in range(1, 500000)]
    # print(len(squares))
```

Output:-

```
Count of squares: 499999
```

Justification:-

Generators yield values lazily on-demand, avoiding storage of all n squares in memory (O(1) space vs O(n)), ideal for large n like 500,000 to prevent memory overflow.Iteration over generator is comparably fast to lists here but scales better under memory pressure.